

AUTOSAR – the Standardized Software Architecture

Stefan Bunzel

Introduction

Mastering the complexity of Electric/Electronics (E/E) architectures is one of the challenges of the automotive industry. In order to manage the growing system complexity and increasing number of dependencies while keeping the costs feasible, the basic software and the interfaces to applications and bus-systems have to be standardized in a future-proof way. In this context, leading automotive manufacturers and suppliers launched the AUTOSAR (AUTomotive Open System ARchitecture) Development Partnership in 2003 [5].

The objective of this global cooperation was to establish an open industry standard for automotive software architecture for suppliers and manufacturers according to the motto: “Cooperate on standards, compete on implementation” [9]. Accordingly the AUTOSAR standard comprises a set of specifications describing software architecture components and defining their interfaces as well as the definition of a standardized development methodology [7]. The AUTOSAR layered software architecture enables the application of independent software components. It aims to increase the reuse of software components, in particular between different vehicle platforms, and between OEMs and suppliers. It enables the scalability of embedded automotive software to different vehicle and platform variants, the transferability of functions throughout the vehicle network, and the integration of functional modules from multiple suppliers. Therefore, the AUTOSAR standard defines an architecture that separates application software from infrastructure-related basic software, as depicted in Fig. 1.

The functional contents of the application software are different and related to the brand identity and the desired characteristics of the car manufacturer, or its system suppliers, whereas the functionality of the basic software is not visible to the customer and thus could be standardized by AUTOSAR in detail.

AUTOSAR Architecture

The layered architecture of AUTOSAR offers all the mechanisms needed for software and hardware independence. It distinguishes between three main software layers which run on a Microcontroller (μ C): Application Software, Runtime Environment (RTE) [4], and Basic Software (BSW) [1], Fig. 2.

The AUTOSAR Basic Software is divided into the following BSW layers: Services, ECU abstraction, microcontroller abstraction, and complex drivers. The Basic Software layers are further divided into functional groups, for example System, Memory and Communication Services.

The complex drivers layer spans from the hardware to the Runtime Environment. It provides the possibility to integrate special-purpose functionality, for instance drivers for devices which are not

DOI 10.1007/s00287-010-0506-7
© Springer-Verlag 2010

Stefan Bunzel
Continental Systems & Technology Automotive,
Guerickestr. 7, 60488 Frankfurt/Main
E-Mail: stefan.bunzel@continental-corporation.com

*Vorschläge an Prof. Dr. Frank Puppe
<puppe@informatik.uni-wuerzburg.de> oder
Prof. Dr. Dieter Steinbauer <dieter.steinbauer@schufa.de>

Alle „Aktuellen Schlagwörter“ seit 1988 finden Sie unter:
www.ai-wuerzburg.de/as

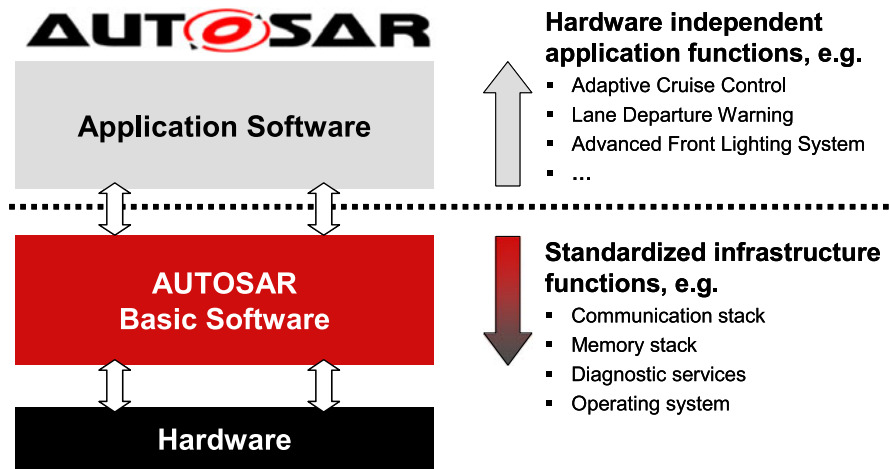


Fig. 1 Application software separated from infrastructure functions

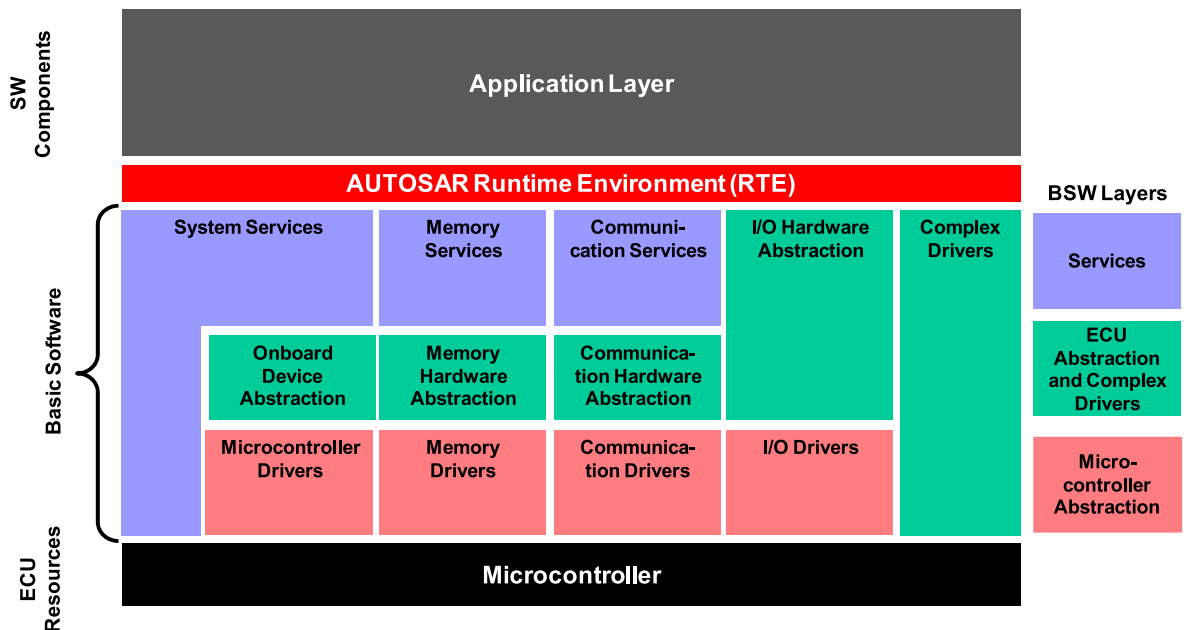


Fig. 2 AUTOSAR Layered Software Architecture

specified within AUTOSAR, having very high timing constraints, or are implemented for migration purposes. The Runtime Environment abstracts the application layer from the basic software and organizes the data and information traffic between them. Above the RTE the software architecture style changes from “layered” to “component” style. The application layer above the RTE is divided into software components. All software components are completely ECU-independent. The defined transfer interfaces of the RTE allow application software components to be developed without specific knowledge of what hardware will be used later. Also, existing

software components can be transferred between ECUs freely.

Although the memory footprint of AUTOSAR software for example depends on the detailed implementation, the contained application functions and their required basic software services, appropriate ECUs need at least 16 bit architecture. Currently the leanest AUTOSAR ECUs on the market seem to require about 256 kB ROM or flash memory.

AUTOSAR Methodology

In addition to defining architecture and interfaces, AUTOSAR also includes a development methodol-

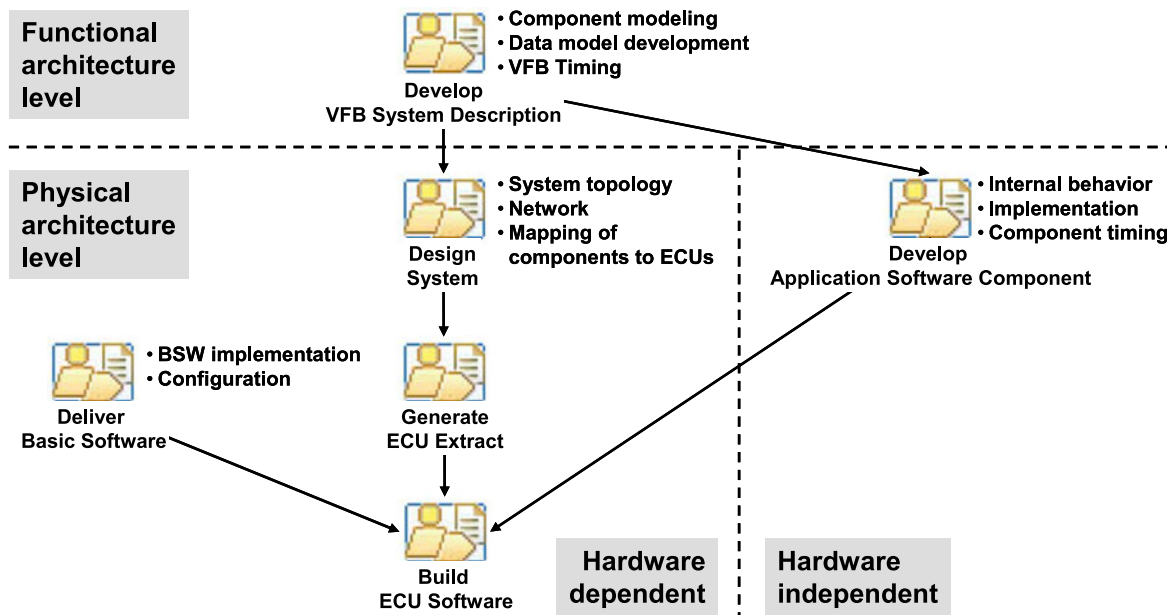


Fig. 3 Methodology overview

ogy description [2, 6]. The AUTOSAR methodology describes the major development steps of an overall AUTOSAR system, which means the entire software for a network of interconnected ECUs in a vehicle. In AUTOSAR Release 4.0 the methodology is specified as a model yielding a set of HTML documentation. It addresses the wide range of software development from the system-level configuration to the generation of an ECU executable, and it supports a widely decoupled development and implementation of application functionality, as well as a seamless integration and configuration of both, the overall system and its individual ECUs. So the methodology is a guiding framework of how to use the AUTOSAR architecture. The methodology is based on the so-called “meta-model.” The meta-model precisely defines all elements to describe a self-contained AUTOSAR system. It is described in UML by means of a tailored profile. Any AUTOSAR description template is derived from this meta-model. The templates are XML files, usually with the extension “.arxml.” Figure 3 depicts an abstract top-level view of the methodology.

The functional architecture level deals with the Virtual Functional Bus (VFB) which enables the development of the functional architecture of the entire system independent from the actual hardware topology of ECUs and the network. It is a partition-

ing or clustering of the application functions into components, i. e. the so-called AUTOSAR software components (SWC). In order to formally handle and model SWCs and their interaction, each SWC needs a formal description: the SWC description.

The next level considers the physical architecture of the entire system, i. e. the activity “design system” leads to a system description that defines the system topology of ECUs, the network, and the mapping of components to ECUs. Before the software for each ECU can be built, the information regarding this ECU has to be extracted from the system description. This allows building and integrating the software for each ECU separately from other ECUs. Of course, building the ECU software requires appropriate basic software for the ECU and all application software components mapped to the ECU.

By now there is a broad variety of basic software implementations from different vendors for many hardware platforms on the market, so that the basic software and corresponding configuration tools can be regarded as off-the-shelf products. The development of the application software components with the definition of the internal behavior, coding, and implementation are independent from hardware and can be done separately for each component. In particular many model-based design

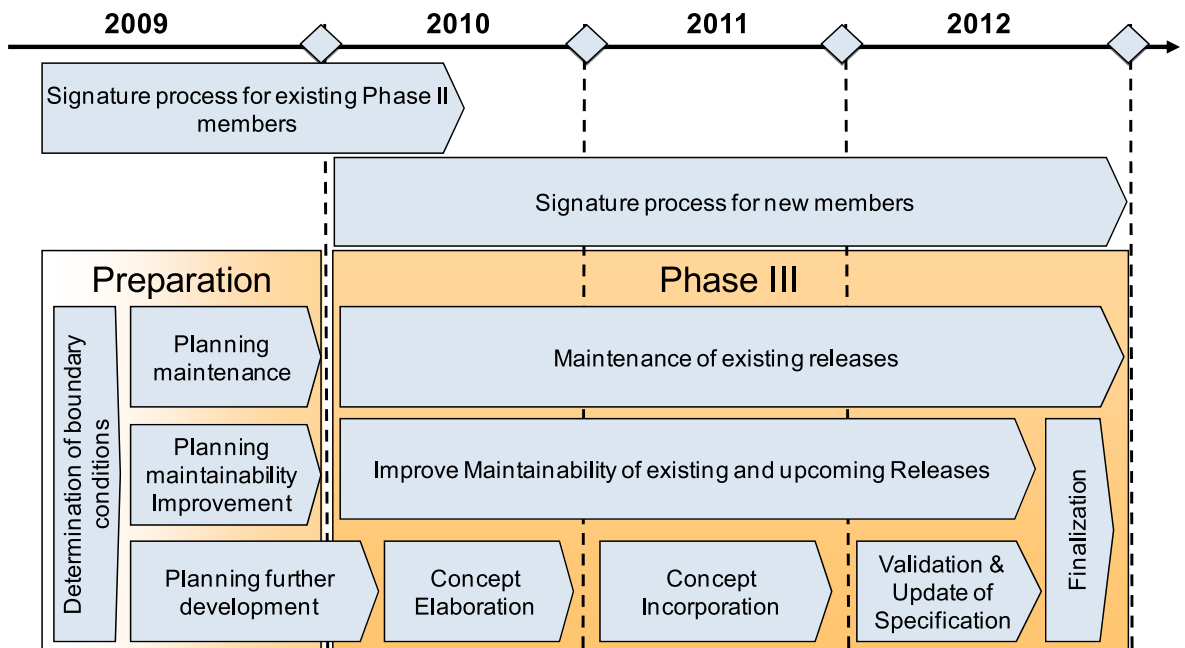


Fig. 4 Draft of Project Roadmap AUTOSAR Phase III

tools on the market can already handle for example the SWC description and thus address the AUTOSAR methodology.

Application Interfaces Support Software Sharing

AUTOSAR supports software sharing by providing architecture, infrastructure, methodology, and the basic software. AUTOSAR simplifies the combination of software by different providers. The baseline for such a scenario is the nonexclusive usage of AUTOSAR basic software provided by the host ECU. Software can be shared by vehicle platforms of the same OEM. In addition, standardized application interfaces support the exchangeability of software between suppliers. AUTOSAR specifies the interfaces of mature and often used applications from all automotive domains regarding their syntax and semantics [3]. This specification is aggregated in a common table, which contains by now nearly 2500 different ports and more than 500 interfaces. These are clustered into about 40 different compositions, and they use 770 data types and 26 units.

Achievements of AUTOSAR Phase I and Phase II

For AUTOSAR Phase II (2007–2009) three releases were planned, providing a continuous improve-

ment of the specifications and introduction of new concepts. Additionally, the experiences from the validation at the end of Phase I and generally considered feedback from all AUTOSAR stakeholders were incorporated. Release 3.0 was published early 2008 and included a large number of improvements, for example the harmonization of the ECU wake-up and the network start-up, and corrections with respect to the previous releases. Release 3.1 was dedicated to the incorporation of On-Board-Diagnostics (OBD) regulations support mechanisms. At the end of Phase II Release 4.0 integrated many new features, for example related to functional safety or communication, and was delivered by the end of 2009 after its validation. Conformance test specifications followed in an update of Release 4.0 by the end of 2010. In order to cover all vehicle functionalities, AUTOSAR has started working on two new car domains in the area of standardized application interfaces during Phase II including Telematics/Multimedia/HMI and Occupants and Pedestrian Safety Systems. Moreover, Powertrain, Chassis, and Body and Comfort interfaces have performed their first steps of integration. In the MM/T/HMI domain the focus was on HMI-related application interfaces. A generic approach enables the separation of HMI-related functions into (a) a pure application service component – independent of the look and feel, (b) user

interface device components, for example for buttons, joysticks, etc., and (c) an application controller, which is responsible for both, the HMI logic and the connection to the user interface. In the occupant and pedestrian safety domain the standardized interfaces belong to the eight compositions: sensor pool, actuator pool, seat belt reminder, vehicle crash detection, occupant restraint system detection, pedestrian protection crash detection, pedestrian protection system activation, and occupant detection.

AUTOSAR Phase III (2010–2012) Prospects

After having developed the standard specifying mature software architecture, AUTOSAR will selectively enhance the standard by adding specific features. Additionally, maintenance of the different releases used for series production will continue in order to support the distribution of AUTOSAR into the market. Consequently, AUTOSAR Phase III will center on one major release, Release 4.1, planned for end 2012 [8]. The proposed concepts for Release 4.1 are going to be elaborated and approved within a dedicated project phase similar to that for Release 4.0. The approved concepts will jointly be worked out in 2010, and subsequently in 2011 incorporated to the specifications, Fig. 4.

Maintenance and selective additions to the standard are the main topics for Phase III. The additions will be specified in such a way that it is possible to ensure backward compatibility wherever feasible and/or to make reliable compatibility statements. In general, the additions shall support new technologies and trends, and shall extend existing functions. The detailed content will be defined during the concept phase of Release 4.1.

Conclusion

As a result of its activities AUTOSAR has already provided several Releases, which comprise a set of specifications describing software architecture components and defining their interfaces. With Release 2.1 and Release 3.0/3.1 the majority of partners and members started their series roll-out of AUTOSAR. In order to ensure a smooth integration of basic software into a vehicle's architecture, an AUTOSAR conformance test system has been defined. It addresses most basic software modules. The test will be conducted by accredited conformance test agencies. When introducing the AUTOSAR standard in series products dedicated migration scenarios need to be applied. Release 4.0 was the next big step for AUTOSAR in providing the features that were demanded by the different applications of the domains covered by AUTOSAR. The AUTOSAR community starts its implementation in 2010. The migration plans of the AUTOSAR Core Partners and Members are proving that it will become the standard for E/E systems in the automotive domain.

References

1. AUTOSAR Layered Software Architecture, R4.0. http://www.autosar.org/download/R4.0/AUTOSAR_EXP_LayeredSoftwareArchitecture.pdf, last access 16.11.2010
2. AUTOSAR Methodology, R4.0. http://www.autosar.org/download/R4.0/AUTOSAR_MOD_MethodologyHTML.zip, last access 16.11.2010
3. AUTOSAR Table of Application Interfaces, R4.0. http://www.autosar.org/download/R4.0/AUTOSAR_MOD_AITable.zip, last access 16.11.2010
4. AUTOSAR Specification of RTE, R4.0. http://www.autosar.org/download/R4.0/AUTOSAR_SWS_RTE.pdf, last access 16.11.2010
5. AUTOSAR Development Partnership Homepage. <http://www.autosar.org>, last access 16.11.2010
6. Bunzel S, Fennel H (2008) The AUTOSAR Methodology. In: VDI (ed) FISITA World Automotive Congress, September 2008, FISITA 2008/F2008-10-023, Springer Automotive Media
7. Fennel H et al (2006) Achievements and Exploitation of the AUTOSAR Development Partnership. SAE Convergence Congress, Detroit
8. Fürst S et al (2009) AUTOSAR – a Worldwide Standard is on the Road. 14th International VDI Congress Electronic Systems for Vehicles 2009, Baden-Baden
9. Heinecke H et al (2006) AUTOSAR – current results and preparations for exploitation. Euroforum conference, May 3rd 2006